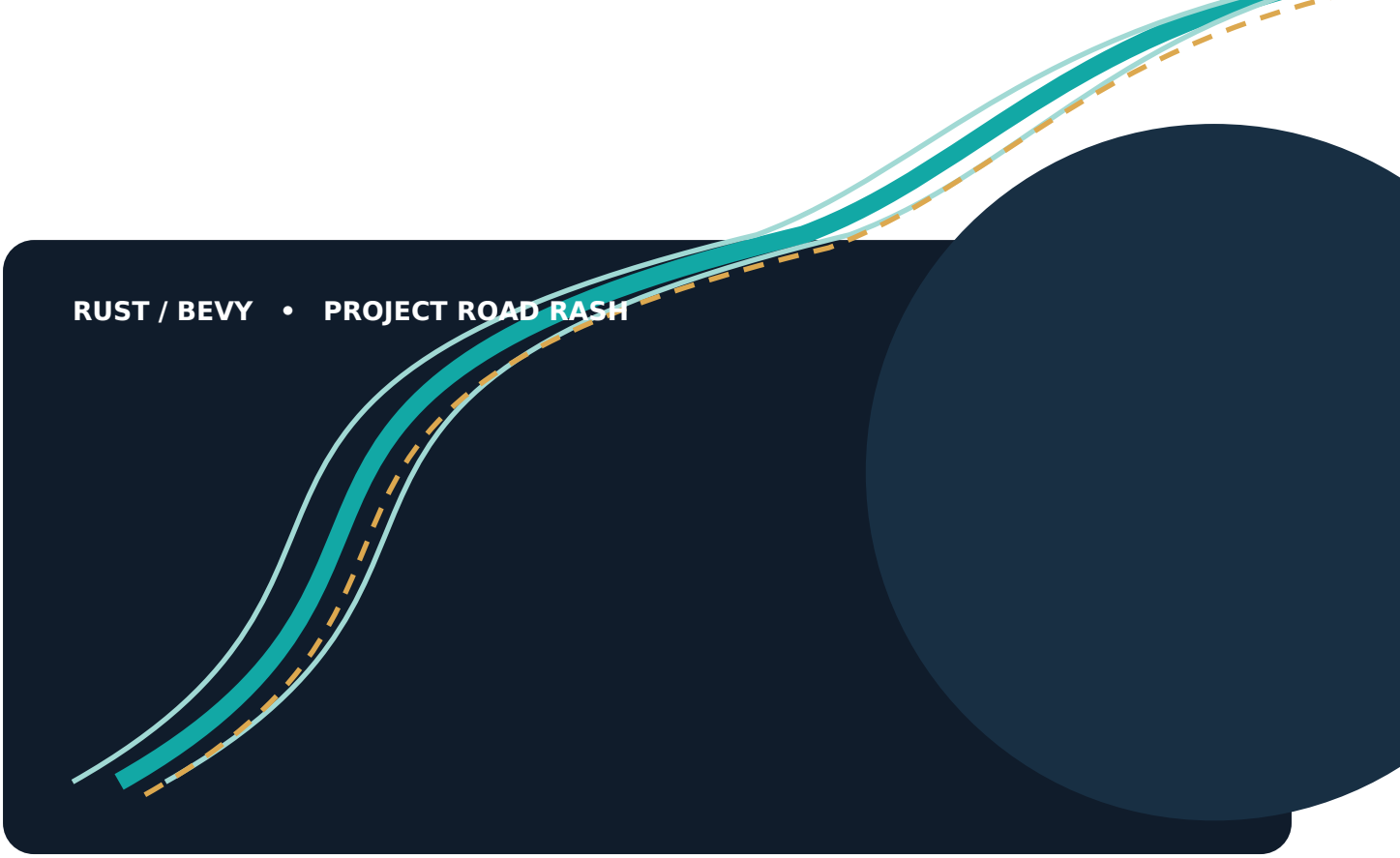


ROAD RASH

Modern Remake & Expansion

Production architecture, 17 build parts, exact GitHub links, allowed reuse and legal boundaries.

DESIGN TARGET	TECHNICAL BASELINE	DELIVERY PHILOSOPHY
PC: Fedora/Linux + Windows	Rust / Bevy 0.20.x / Avian3D 0.8	Faithful gameplay identity; new technology
Solo-first, team-scalable	Lightyear 0.31 for later multiplayer	Original assets or licensed IP only
Offline vertical slice first	60 Hz simulation / 60-120 FPS render	Quality gates before new content

Critical framing

The GitHub Road Rash port is a behavioral research reference, not a code base to port. The new game uses original Rust, 3D models and modern physics. First-class priorities: lively towns/farms/cliffs with memorable hazards, fully controllable on-foot recovery (including walking across the finish line), and a legal free-audio pipeline.

Version 3.0 | Prepared October 30, 2026 | Technical choices reflect public documentation checked on that date.

READ THIS BEFORE STARTING

Executive decisions

The one-sentence plan

First prove a fast, fun motorcycle-combat race on an interesting road with farms, towns, dramatic cliffs and memorable hazards. Preserve controllable on-foot recovery and foot-finish exploits-as-fun. Then harden the systems, build a world-content pipeline, introduce multiplayer, and scale original high-quality scenery, audio and career content.

Scope boundary

- In scope: third-person motorcycle combat racing, five thematic environments over time, damage and crashes, AI, traffic, police, career/shop progression, 3D visuals, Linux/Windows, online play, controller support.
- Not in the first playable: exact original maps, full career, 15 finished bike models, eight-player networking, photorealism, weather systems, character customization, workshop/modding. Core crash-to-foot control and crossing the finish line without the bike ARE in scope.
- Legal default: build as an original spiritual successor unless a negotiated EA license explicitly authorizes Road Rash branding, exact courses/characters/art, soundtrack or other protected material.
- Philosophy: use old Road Rash as a design target, not an engine specification; do not attempt deterministic frame matching to the 1996 binary.

Current verified dependency pins (research baseline)

Component	Baseline	Reason / note
Bevy	0.20.1 patch line	Engine / ECS / rendering / input, verify lockfile
Avian3D	0.8.0	Physics, rigid bodies, raycasts, collision queries; supports Bevy 0.20
Lightyear	0.31.0	Server authority / prediction / snapshots; supports Bevy 0.20
bevy_enhanced_input	0.27.x	Action mappings, controllers, deadzones; supports Bevy 0.20
Bevy Hanabi	main branch supports 0.20	Optional GPU particles; pin a known commit/release
Blender + glTF 2.0	Stable tested build	Model/rig/animation/scene authoring, exports

Pinned values are a starting point, not an instruction to mix unmatched plugin releases. The repository lockfile and a passing Linux + Windows build define the actual supported combination. Sources [S01]-[S06].

PART MAP

The sequence at a glance

Part	Build area	Working output / exit artifact
00	Preproduction, rights & research	Original-content policy, reference database, scope locked
01	Workspace, CI, conventions	Fedora/Windows launch + tests + debug overlay
02	Scenic road/track graybox	Driveable test road linking town, farms and cliffs
03	Bike physics & controls	Three distinct handling presets and automated tests
04	Camera + moment-to-moment feel	Readable 3rd-person fast riding
05	Combat foundations	Rider-to-rider punches/kicks register reliably

Part	Build area	Working output / exit artifact
06	Crashes + controllable on-foot recovery	Ragdoll -> auto-return/manual foot control -> remount
07	Race rules + foot-finishing	Bike OR rider can legally cross finish gate
08	AI, civilian traffic, police	Fun 4-rider + traffic test race
09	Vertical slice / playtest gate	One repeatable, enjoyable 3-minute race
10	Networking feasibility spike	2-4 player latency/loss prototype
11	Scenic world production	Cities, farms, animals, cliffs, props and licenses
12	Progression / garage / save	Cash, garage, bike upgrades, save/load
13	Full multiplayer + lobby	Authoritative online race + results
14	Content production + expansion	New tracks, roster, bikes, modes
15	Graphics, audio systems and UX	Free-audio pipeline + spatial environments + polish
16	Profiling, QA, shipping	Windows/Linux builds, Steam-ready submission

Do not parallelize too early

Finish the driving feel and one enjoyable offline race before investing in dozens of polished assets. Do a small networking spike immediately after the vertical slice, before making irreversible physics and animation decisions.

REPOSITORY USAGE CONTRACTS

GITHUB REPOSITORIES - START HERE

Read this before you write code

These are the nine exact GitHub repositories in the production plan, with clickable URLs, roles, licensing boundaries, and the parts where they enter. We are not building on the C++ Road Rash port. We write the game in Bevy and use the original game only to learn what made it fun.

A. Repositories we INSTALL as our game dependencies

R08 | Bevy engine - Part 01 onward

https://github.com/bevyengine/bevy

- What goes into our game: Core ECS, 3D renderer, transforms, scheduling, asset loading, audio baseline.
- Integration rule: Direct engine dependency; use official examples. MIT OR Apache-2.0.

R04 | Avian3D physics - Parts 01-08, 10, 13

https://github.com/avianphysics/avian

- What goes into our game: Colliders, bike rigid body, tire contact queries, collision response, crash ragdolls, CCD.
- Integration rule: Direct dependency: Avian3D 0.8.x for Bevy 0.20; MIT OR Apache-2.0.

R06 | Bevy Enhanced Input - Parts 01, 03-06, 13

https://github.com/simgine/bevy_enhanced_input

- What goes into our game: Throttle/brake/steering, attack inputs, manual on-foot movement, controller rebinding.
- Integration rule: Direct dependency: 0.27.x for Bevy 0.20; MIT OR Apache-2.0.

R05 | Lightyear - Parts 10, 13

https://github.com/cBournhonesque/lightyear

- What goes into our game: Client prediction, authoritative server, replication, interpolation, combat event sync.
- Integration rule: Add at networking spike, not day one. 0.31 for Bevy 0.20; MIT OR Apache-2.0.

R09 | Bevy Kira Audio - Part 15; optional earlier

https://github.com/niklasei/bevy_kira_audio

- What goes into our game: Layered motorcycle RPM loops, music/ambience buses, fades, pitch control, positional SFX.
- Integration rule: OPTIONAL: first test Bevy built-in audio. If using Kira, disable Bevy default audio feature; 0.27 for Bevy 0.20.

R07 | Bevy Hanabi - Parts 11, 15

https://github.com/djeedai/bevy_hanabi

- What goes into our game: Dust, exhaust, sparks, tire skids, environmental particle effects.
- Integration rule: OPTIONAL: pin an explicitly compatible Bevy 0.20 commit/release; MIT OR Apache-2.0.

Dependency selection

Install Bevy + Avian + Enhanced Input in the first bootstrap. Wait until a proven offline vertical slice to add Lightyear. Kira and Hanabi are optional: use only if built-in Bevy systems fail the audio/VFX acceptance tests.

NO HIDDEN CODE REUSE

GITHUB REPOSITORIES - RESEARCH / EXAMPLES

B. Repositories to STUDY - we will not port them

R01 | arkits/road-rash - Parts 00, 02, 03, 05, 07, 08, 12

https://github.com/arkits/road-rash

- Precisely what we extract as knowledge: 1996 rules, bike classes, track geography, AI racer behavior, character/color information, combat pace, shops and score mechanics.
- Boundary: Research only. No verified permission to reuse implementation; original EA assets/branding carry independent rights. Study docs and behavior, then write original Rust.

R03 | blyfoten/driftmaster - Parts 03, 06

https://github.com/blyfoten/driftmaster

- Precisely what we extract as knowledge: Read src/physics.rs and tests to understand tire friction, chassis stability, raycast suspension, telemetry and behavior tests.
- Boundary: Research only while code license is unverified. Written for Bevy 0.14; do not plug it into a Bevy 0.20 game.

R02 | Bevy-Jam-5/bike-game (Crazy Bike) - Parts 00, 02, 03, 04, 11

https://github.com/Bevy-Jam-5/bike-game

- Precisely what we extract as knowledge: Study bike control, 3D scenery, pedestrian-scale objects, third-person camera, physics interactions, and scene/asset assembly.
- Boundary: Code explicitly MIT OR Apache-2.0; may adapt with notices if useful. Third-party city pack and music/sounds carry mixed licenses. Do NOT bulk-copy its assets.

C. Do NOT confuse GitHub code with free media catalogs

World art, cattle/farm models, building meshes, asphalt materials, motorcycle engines, cow calls, rock music, impact SFX and atmosphere are separate content acquisitions. The existing sections on Poly Haven, Mixkit, Kenney, Sonniss and OpenGameArt identify candidate sources; these are not automatically code dependencies. Every downloaded file needs a recorded license, attribution and permitted redistribution status.

How we credit and audit third parties

Maintain `docs/third_party/REPOSITORIES.md` (repo, selected version/commit, purpose, license, notices) and `docs/third_party/ASSETS.csv` (source URL, author, exact sound/model file, license, whether shipped). Only the dependencies and vetted art/audio assets actually used appear in shipped notices.

FIND EACH REPO WHERE WE USE IT

Repository-to-development-part map

This map is the practical order. Every Part 00-16 chapter also includes its own clickable GitHub links under EXACT GitHub repositories for this part.

Part	What we build	Repos to open now
00	Reference study, rights	R01 Road Rash, R02 Crazy Bike
01	Rust workspace + empty Bevy scene	R08 Bevy, R04 Avian, R06 Enhanced Input
02	Road graph, slopes, town/farm/cliff graybox	R08 Bevy, R02 Crazy Bike, R01 Road Rash
03	New motorcycle chassis and tire handling	R04 Avian, R02 Crazy Bike, R03 DriftMaster
04	Camera, controls, sense of speed	R08 Bevy, R06 Enhanced Input, R02 Crazy Bike
05	Riding melee / weapons	R04 Avian, R01 Road Rash
06	Crash, ragdoll, independent on-foot rider	R04 Avian, R03 DriftMaster
07	Race, finish on foot, ranking, HUD	R08 Bevy, R01 Road Rash
08	AI racers, road traffic, police	R01 Road Rash, R04 Avian
09	Vertical slice fun gate	R08 Bevy, R02 Crazy Bike
10	Netcode feasibility spike	R05 Lightyear, R04 Avian
11	Detailed towns, farms, cows, cliffs	R08 Bevy, R02 Crazy Bike, R07 Hanabi (optional)
12	Garage, progression, data/save	R01 Road Rash, R08 Bevy
13	Multiplayer, hit validation, lobbies	R05 Lightyear, R04 Avian, R06 Enhanced Input
14	Content and world expansion	R08 Bevy, R02 Crazy Bike
15	RPM engine audio, ambiance, particles	R09 Kira (optional), R07 Hanabi (optional)
16	Profiling, license audit, release	R08 Bevy, R04 Avian, R05 Lightyear

First implementation move

In Part 01, use Bevy + Avian3D + Enhanced Input. In Part 02, build a road through a town, a cattle farm and coastal cliffs. In Parts 03-07, build modern bike dynamics, combat, crash-to-foot control and a valid on-foot finish. Do not add Lightyear, Kira or Hanabi until their designated gates.

CONTRACTS BEFORE CODE

Technical architecture

Runtime responsibilities

Module	Owned state / responsibility	Strict boundary
client	Input, render state, camera, FX, UI, local prediction	Never adjudicates a race or a confirmed hit
sim	Bike motion; AI; combat requests; race state; deterministic-ish fixed rules	No renderer or UI dependencies in pure-domain modules
physics	Avian adapter, collision scene, contacts, raycasts, impulses	Physics state observed via explicit components/events
server	Authoritative tick, inputs, hits, rankings, prizes, anti-cheat	No game outcome from client-reported transforms
content	Typed bike/rider/weapon/track definitions; schema versions	No hardcoded tuning constants scattered across systems
tools	Track validation; content checks; profiling; replay inspection	No production gameplay dependency on dev-only tools

Suggested Rust workspace

```
road-rash-modern/
  Cargo.toml                  # workspace, pinned dependency versions
  crates/
    game_client/              # Bevy app, rendering, UI, camera
    game_server/              # authoritative headless host
    rr_core/                  # shared IDs, events, state, math helpers
    rr_vehicle/               # new motorcycle dynamics & tuning
    rr_combat/                # attacks, validation, hit reactions
    rr_race/                  # checkpoints, placement, race rules
    rr_ai/                    # racers, traffic, police states
    rr_content/               # serde definitions and validators
    rr_network/               # Lightyear protocol and replication
    rr_tools/                 # debug, perf, asset validation
  assets/{tracks,bikes,riders,fx,audio,ui}/
  content/{bikes,riders,weapons,tracks}/
  docs/{research,decisions,licenses,tests}/
  .github/workflows/
```

Principles that prevent rewrites

- Fixed simulation: 60 Hz default; interpolate renders independently. Instrument the actual physics schedule rather than assuming a numeric tick count makes it deterministic.
- Data first: stable string/UUID IDs for bikes, weapons and riders. Separate cosmetic variant IDs from physics tuning IDs.
- Event flow: inputs -> intent -> simulation -> authoritative events -> visuals/audio; avoid animations directly changing outcomes.
- One source of truth: authoritative race state and results; game UI only derives displays.
- Feature flags: `devtools`, `debug_physics`, `offline`, `netplay`, optional quality modes; reproducible builds.
- Migration-safe: lock versions and define serialization schema version on save files and network protocol.

BUILD SEQUENCE

Part 00 | Research, scope and legal foundation

Goal

Write a short GDD, protect the project legally, and build a traceable content reference system without importing original assets.

Implementation order

- 01. Declare whether this is an internally researched original successor or a licensed Road Rash remake; presume original successor for public distribution until rights are secured.
- 02. Record the classic pillars: speed, combat, crashes, traffic, police, bike upgrades. Define exact player fantasy and target session length.
- 03. Create a research ledger: every observation receives source URL, version, confidence, screenshot/recording description, and reimplementation note.
- 04. Build content sheets for bike properties, weapons, rider styles, outfits, palettes, tracks, shop items, progression, UI.
- 05. Define an MVP: 1 road, 1 bike + 2 variants, 3 AI opponents, punches/kicks, traffic, finish/results/retry; no original protected media.
- 06. Maintain a rights register for each art/audio/font/third-party dependency with origin, owner, license and export status.

Engineering specification

- GDD v0.1 under 12 pages; each feature tagged P0/P1/P2; explicit performance and fun metrics.
- Research `source_id`, `observed_behavior`, `confidence`, `asset_owner`, `allowed_usage`, `follow_up` columns.
- No verbatim copy of proprietary code or extracted art; verify README and individual file licenses before reuse.

Validation / tests

- At least 20 reference observations documented with confidence levels.
- Legal inventory has no unknown-license content in runtime assets.

Exit gate: do not advance until

Definition of done

An original-content policy, content schema, top-level backlog, and one-page vertical-slice definition all exist.

EXACT GitHub repositories for this part (click to open)

- [R01] arkits/road-rash - Research bike tables, character and AI discoveries, progression and course behaviors; do not ingest C++ as a source port.
  https://github.com/arkits/road-rash | RESEARCH ONLY - no source/asset reuse assumed
- [R02] bike-game - Study how a small Bevy project structures game-loop systems and scene entities.
  https://github.com/Bevy-Jam-5/bike-game | LICENSED EXAMPLE - MIT/Apache code; audit assets separately

Watch-outs

The more closely final art, track layouts, named characters and branding reproduce EA expression, the greater the clearance need - even if all source code is original.

BUILD SEQUENCE

Part 01 | Bootstrap the Rust / Bevy project

Goal

A reliable, reproducible project that launches on Fedora KDE and produces Windows builds.

Implementation order

- 01. Create Rust workspace, `rust-toolchain.toml`, locked dependencies, `clippy`, `rustfmt`, deny/warnings policy and conventional commit rules.
- 02. Create an app state machine (Boot, MainMenu, Loading, Race, Results), a plugin per subsystem and a fixed-step schedule.
- 03. Set render diagnostics (FPS, frame-time percentiles), physics diagnostics, on-screen debug menu, logging and panic reports.
- 04. Build empty 3D scene with a ground plane, directional light, primitive camera, interactive pause/reset and input map.
- 05. Configure CI matrix: cargo fmt, clippy, test, Linux build, Windows cross-build when validated, license/asset audit.

Engineering specification

- Bevy 0.20.1, avian3d 0.8.0, bevy_enhanced_input 0.27; pin exact patch resolution in Cargo.lock.
- Use `FixedUpdate` for gameplay; reset/pause must suspend simulation without corrupting state.
- Metrics: 60Hz tick counter, avg/P95 frame-time, entity count, colliders, physics stepping time; a toggleable overlay.

Validation / tests

- `cargo test --workspace` passes.
- A release-mode native build starts on Fedora with NVIDIA driver and on a Windows test environment.
- App survives 100 pause/reset cycles.

Exit gate: do not advance until

Definition of done

Blank scene launches and measures 60 Hz game ticks independently of render FPS; CI succeeds.

EXACT GitHub repositories for this part (click to open)

- [R08] bevy - Use official 0.20 examples for ECS, schedules, cameras and rendering.
  https://github.com/bevyengine/bevy | CORE ENGINE DEPENDENCY - MIT/Apache
- [R04] avian - Add physics only with confirmed compatible API.
  https://github.com/avianphysics/avian | DIRECT DEPENDENCY - MIT/Apache
- [R06] bevy_enhanced_input - Use action mapping examples for remappable keys and controller support.
  https://github.com/simgine/bevy_enhanced_input | DIRECT DEPENDENCY - MIT/Apache

BUILD SEQUENCE

Part 02 | Build a road graybox and track format

Goal

One visually plain but driveable road, reliable collision geometry, start/finish checkpoints and authorable turns.

Implementation order

- 01. Choose coordinate convention: meters, +Y up, forward along spline tangent. Document units and origin.
- 02. Implement Catmull-Rom/Bezier centerline spline with lanes, widths, banking, elevation and sector markers.
- 03. Generate coarse road mesh, shoulders, guardrails and simplified collision mesh; test curvature limits and intersections.
- 04. Add checkpoint gates and route progress mapping to a canonical centerline distance.
- 05. Create a track inspector: spline gizmos, normals, triangle count, driving line, bounding areas and spawn positions.
- 06. Build a short 2 km road for physics iteration, then a roughly 6-9 km scenic race corridor (tune to a ~3-minute average race): town streets -> rural farms -> winding cliffside section. Include long straight, turns, hairpin, and roadside hazards.

Engineering specification

- Track schema contains `id`, control points, lane width, surface ID, optional split/merge, race checkpoint IDs and prop placement.
- Build collision representation lower resolution than render mesh; 2-3 meter segment sampling as an initial adjustable target. Road metadata includes shoulders, curbs, fencing, walkable/verge regions, animal-crossing and cliff hazard zones.
- Use surface metadata: asphalt, dirt, gravel, grass, water; default friction and traction modifiers.
- Validate checkpoint order and spawn overlap; support origin rebase if future tracks require it.

Validation / tests

- Track importer rejects NaNs, self-crossing checkpoints and negative width.
- Invisible walls and highway guardrails prevent falling through geometry.
- Track loads headlessly for validation.

Exit gate: do not advance until

Definition of done

Graybox course supports a complete closed-loop or point-to-point race with clear progress measurement.

EXACT GitHub repositories for this part (click to open)

- [R01] arkits/road-rash - Research original road pacing, curvature, splits and environmental hazard placement - do not copy protected track data.
  https://github.com/arkits/road-rash | RESEARCH ONLY - no source/asset reuse assumed
- [R08] bevy - Procedural meshes and glTF scene loading patterns.
  https://github.com/bevyengine/bevy | CORE ENGINE DEPENDENCY - MIT/Apache

BUILD SEQUENCE

Part 03 | Modern arcade motorcycle controller

Goal

The most important prototype: satisfying riding that feels fast and controllable rather than a rigid-body science project.

Implementation order

- 01. Implement a dynamic bike root body with center of mass, contact points and stable upright assist; visually animate rider and wheels separately.
- 02. Build throttle, engine braking, service brakes, steering, lean target, tire slip, tire friction per surface and speed-sensitive steering.
- 03. Start with assisted 1-body controller + two wheel contact raycasts; do NOT require a full multibody motorcycle before testing fun.
- 04. Add traction-limited acceleration, turn-in, sliding, collision response and self-righting at low speed; expose tuning live in a panel.
- 05. Create three presets - light/agile, balanced, heavy/powerful - from one data structure; keep mass and torque physically plausible but player-friendly.
- 06. Create automated acceleration, brake-distance, steady-turn, lean-recovery and curb-hit benchmarks; log 100Hz trace where needed.

Engineering specification

- Initial tuning targets (not original Road Rash facts): 0-100 km/h around 3.5-6.5 seconds by class; lean clamp near 50-60 deg; 60Hz fixed step; adjustable maximum steer/assist.
- Contact forces: spring-damper suspension; longitudinal/lateral grip envelope; longitudinal drag ~v^2; optional gyroscopic stabilization approximation.
- Physics: Avian CCD enabled selectively for high-speed hazards; dynamic body, filtered collider groups; contact callbacks routed into damage events.
- Simulation-facing `BikeState`: position, velocity, yaw/lean, slip, grounded wheels, engine RPM, gear, stamina/health references.

Validation / tests

- Test acceleration and deceleration with numeric tolerances at 30/60/120 render FPS.
- Bike survives 10 minutes of sharp turning without NaNs, tunneling or unexplained flipping.
- Record at least 10 test rides and compare perceived handling.

Exit gate: do not advance until

Definition of done

3 presets are clearly distinct, a beginner can finish the course, and handling remains fun with placeholder art.

EXACT GitHub repositories for this part (click to open)

- [R02] bike-game - Study 3D bike control and contact responses; licensed code only if needed and properly attributed.
  https://github.com/Bevy-Jam-5/bike-game | LICENSED EXAMPLE - MIT/Apache code; audit assets separately
- [R03] driftmaster - Research tire force calculations, raycast suspension and telemetry; independently implement.
  https://github.com/blyfoten/driftmaster | RESEARCH ONLY - permission to reuse code not verified
- [R04] avian - Use actual physics solver, CCD and raycast APIs.
  https://github.com/avianphysics/avian | DIRECT DEPENDENCY - MIT/Apache

Watch-outs

Do not confuse realistic banking with fun; an assisted lean target is a gameplay tool. Network prediction of a complex physics solver may diverge, so isolate this controller behind a state-update contract now.

BUILD SEQUENCE

Part 04 | Chase camera, speed sensation and controls

Goal

Make the bike readable and exhilarating across keyboard and gamepad before developing the rest of the game.

Implementation order

- 01. Build spring-follow chase camera, anticipation based on heading/turn input, look-ahead point, obstacle avoidance and collision zoom.
- 02. Add optional near and far chase cameras; controller right-stick freelook; a basic accessibility FOV/smoothing panel.
- 03. Add speed-dependent audio pitch and wind noise, mild camera shake, speed lines (optional) and visible slip/traction effects.
- 04. Implement brake/throttle/steering actions; gamepad deadzones, analog triggers, key remapping, pause/reset.
- 05. Test camera and steering together at extreme turns, collisions, jumps and instant respawns.

Engineering specification

- Camera tick uses interpolated pose, not direct fixed-step transforms; no jitter at 120-144Hz.
- Configurable FOV range target 60-95 deg; default around 75 deg with subtle speed change; anti-nausea mode reduces shake.
- Gamepad input: left stick steer, triggers throttle/brake, face buttons attacks, shoulder buttons alternate attacks/looking.

Validation / tests

- No camera clipping into roadside meshes during standard play.
- Keyboard and gamepad finishable; no unintentional auto steering.
- Comfort settings persist.

Exit gate: do not advance until

Definition of done

An external tester says riding feels responsive, fast and readable without needing final art.

EXACT GitHub repositories for this part (click to open)

- [R02] bike-game - Reference bicycle camera and input interactions.
  https://github.com/Bevy-Jam-5/bike-game | LICENSED EXAMPLE - MIT/Apache code; audit assets separately
- [R06] bevy_enhanced_input - Implement binding contexts and deadzone settings.
  https://github.com/simgine/bevy_enhanced_input | DIRECT DEPENDENCY - MIT/Apache

BUILD SEQUENCE

Part 05 | Melee combat while riding

Goal

A predictable, enjoyable Road Rash-style attack loop that interacts with new motorcycle physics.

Implementation order

- 01. Implement attacks as explicit abilities: left punch, right punch, left kick, right kick; add one simple held weapon later.
- 02. Specify per-action startup, active, recovery, range, stamina, impulse, damage, stagger and cooldown fields.
- 03. Determine side/front cone using stable bike/rider transforms; use swept hit detection and server-ready hit events.
- 04. Create rider animation state machine (idle, lean, punch L/R, kick L/R, hit reaction, stagger); synchronize only gameplay-important timing.
- 05. Add attack buffering, input priority and anti-spam; prototype hit sound and concise HUD feedback.
- 06. Write opponent test harness with side-by-side bikes, velocity matching and deterministic attack distances.

Engineering specification

- Example proposed tuning: punches ~0.15s startup/0.12s active/0.3s recover; kicks ~0.24s startup/0.16s active/0.5s recovery; exact numbers are initial design values.
- Combat event: attacker, target candidate, attack_id, contact point, relative speed, tick, result; resolve damage/knockback once.
- Melee hitboxes are transient sensor/swept volumes; avoid untrusted animation-only damage events.

Validation / tests

- A single attack cannot double-hit due to overlapping colliders.
- Attacks missed at 0.5m outside range do not connect; matched-speed side targets within range do.
- Automated tests verify cooldown, stamina and immunity windows.

Exit gate: do not advance until

Definition of done

Fighting other riders is enjoyable for 3 minutes with only punches/kicks and no weapons.

EXACT GitHub repositories for this part (click to open)

- [R01] arkits/road-rash - Reference original weapon categories, relative damage, telegraph and combat pacing.
  https://github.com/arkits/road-rash | RESEARCH ONLY - no source/asset reuse assumed
- [R04] avian - Use shapecasting/contact queries for melee volumes.
  https://github.com/avianphysics/avian | DIRECT DEPENDENCY - MIT/Apache

BUILD SEQUENCE

Part 06 | Damage, ragdoll, crash and recovery

Goal

Generate the signature slapstick crashes without making crashes so frequent or slow that racing stops being fun.

Implementation order

- 01. Implement rider condition (health/stamina), bike durability, wobble/balance and crash thresholds.
- 02. Define crash sources: vehicle impact, opponent attack, curb, roadside obstacle, police hit, airborne bad landing.
- 03. Switch from riding to physical ragdoll on knockoff, settle to a recoverable on-foot state, and allow steering/running anywhere. Default to an automatic run toward the bike only when the player supplies no manual walking direction; manual input must override the assist immediately.
- 04. Spawn sparks, impact audio, debris only from one authoritative crash event; separate cosmetic particles from collision state.
- 05. Define remount trigger at bike proximity, interaction facing and/or automatic proximity; provide optional stuck recovery after a delay. Avoid teleporting automatically across the finish or snapping the rider onto the bike while they are deliberately walking away.
- 06. Tune crash frequency/time loss; make the foot route interesting: crossing road, avoiding traffic and reaching finish line on foot must be possible where valid. Audit fences, buildings and cliffs for on-foot trapping.

Engineering specification

- Initial target: crash -> rider receives manual ground control in ~1-3 seconds (playtest tuned); then on-foot until remount, arrest, death/reset or finish. Auto-return is steering assistance, not forced teleport.
- At least two collider layers: simulation-critical bike/rider/hazards and cosmetic debris; avoid per-fragment replication.
- Ragdoll: 10-18 joints/bodies target only after performance is known; a simplified 5-body ragdoll acceptable for prototype.
- High-speed CCD on fast player chassis only; player on-foot controller uses collision capsule, slope/step filters, terrain and fence masks, and optional character-controller style kinematics for predictable foot handling.

Validation / tests

- 100 repeated high-speed crashes produce no frozen simulation or inverted terrain clipping.
- On-foot manual direction always overrides auto-return; returning to neutral resumes assisted bike seeking. Walking away does not cause unwanted remount or teleport; rider can legally cross finish gate by foot.
- Ragdoll disables or changes collision filters correctly.

Exit gate: do not advance until

Definition of done

Knockoff, ragdoll, independent walking, optional assisted return, manual finish and remount are smooth enough to keep racing fun.

EXACT GitHub repositories for this part (click to open)

- [R04] avian - Collision impulses, joints, CCD, contact filters.
  https://github.com/avianphysics/avian | DIRECT DEPENDENCY - MIT/Apache
- [R03] driftmaster - Reference chassis inertia / stability telemetry, not its source code.
  https://github.com/blyfoten/driftmaster | RESEARCH ONLY - permission to reuse code not verified

BUILD SEQUENCE

Part 07 | Core race rules, HUD and results

Goal

Make a repeatable race from start countdown to finish, rewards and restart.

Implementation order

- 01. Implement countdown, seeded race start, checkpoint progression, finish validation for bike OR rider entity, race placement and out-of-bounds recovery.
- 02. Add clock, speedometer, placement, distance/progress, health/bike damage, weapon indicator and basic minimap strip; switch HUD context when on foot (bike arrow, distance to bike, optional recovery hint).
- 03. Implement results: rank, time, penalties, payout; support quick restart and practice/free-ride.
- 04. Build save-safe race summaries and record best times per bike class/track.
- 05. Add data-driven tuning for race length, AI count, traffic density, police frequency and difficulty.

Engineering specification

- Checkpoint format: ordered forward-crossed gates along road; maintain `last_valid_checkpoint`, `route_segment`, `distance_along_route`, `lap`, `finish_tick`. Rider and bike share racer_id but have separate transforms.
- Placements based on route progress and crossed checkpoints, never pure 3D distance to finish.
- Finish is a racer-level, server-authoritative and idempotent event (not a bike-only collision); a dismounted rider can cross the finish plane with forward progress and valid prior checkpoints. Validate against finish teleport/shortcut exploits.

Validation / tests

- Cannot finish by crossing backward or skipping required checkpoints; rider walking through finish succeeds when prior gates are valid; bike and rider crossing in same tick yields one result.
- Placements stable with split path and temporary off-road movement.
- Retry retains settings but resets all live entity state.

Exit gate: do not advance until

Definition of done

Single 3-minute course playable start-to-results-to-restart with stable outcomes.

EXACT GitHub repositories for this part (click to open)

- [R01] arkits/road-rash - Reference race payouts, UI information priorities and progression logic.
  https://github.com/arkits/road-rash | RESEARCH ONLY - no source/asset reuse assumed
- [R08] bevy - UI and event flow examples.
  https://github.com/bevyengine/bevy | CORE ENGINE DEPENDENCY - MIT/Apache

BUILD SEQUENCE

Part 08 | AI racers, traffic and police

Goal

Populate the road with opponents who race and fight intelligently rather than following identical prerecorded paths.

Implementation order

- 01. Build racing line sampler and simple opponent controller using the same BikeControl interface as a player.
- 02. Add speed planner (curvature + lane + upcoming traffic), overtake states, defense states, attack choice and recovery handling.
- 03. Implement opponent archetypes (aggressive brawler, cautious finisher, opportunistic attacker, reckless speeder) from weights not hard-coded cheats.
- 04. Add civilian cars using road-following and lane-change logic; police use distinct chase/bust trigger and pursuit state machine.
- 05. Design event budgets: max active racers/traffic/police within camera area, spawn/despawn outside safe visibility boundaries.
- 06. Instrument AI decisions with onscreen goal/desired speed/path; add headless tournaments to tune behavior.

Engineering specification

- AI states: Cruise, SeekOvertake, Defend, Attack, Recover, Pursued, Arrested; blackboard with target speed and opponent proximity.
- Never grant infinite acceleration or impossible corner grip to the AI unless explicitly in a marked arcade difficulty mode.
- Traffic uses simplified vehicle physics; full wheel raycasts not required for every traffic car.
- Difficulty parameters should change risk tolerance, braking margins and aggression, not secretly multiply all stats.

Validation / tests

- 100 simulated races complete without deadlocks.
- AI can overtake/hit player and can recover from being knocked off; test AI and human rider-foot states separately.
- Police chase/pursuit starts and ends under testable conditions.

Exit gate: do not advance until

Definition of done

One four-racer race with live traffic, police and melee can be completed repeatedly and remains fair.

EXACT GitHub repositories for this part (click to open)

- [R01] arkits/road-rash - Reference pacing, threat composition and specific observed AI behaviors.
  https://github.com/arkits/road-rash | RESEARCH ONLY - no source/asset reuse assumed
- [R03] driftmaster - Use telemetry design ideas for AI controller testing.
  https://github.com/blyfoten/driftmaster | RESEARCH ONLY - permission to reuse code not verified

BUILD SEQUENCE

Part 09 | Vertical-slice fun gate

Goal

Prove the actual game is fun before creating a huge content library or expanding graphics.

Implementation order

- 01. Consolidate a ~3-minute route with contrasting town/farm/cliff scenery, player + three AI, traffic/police, punch/kick, crashes, independent on-foot controls, foot finish, checkpoint race and results.
- 02. Run at least 10 external play sessions; capture video, death/fail causes, input issues, reaction to crashes and perceived speed.
- 03. Measure engagement: percentage finishing, restart rate, attacks attempted, successful hits, collisions, times spent immobilized.
- 04. Run handling A/B settings and retain favorites; stop adding features until one setting wins.
- 05. Write explicit keep/change/remove decisions and freeze minimal interfaces for online prototype.

Engineering specification

- Initial proposed targets: 80% of first-time testers finish in <=3 tries; >50% voluntary replay; time outside control below 12% of session.
- Prototype test on RTX 3080 Fedora system at 1080p: 60+ FPS P95 baseline with all P0 gameplay active (not final visual quality).

Validation / tests

- All core loops function without debug-console intervention.
- At least 10 honest playtest reports logged.
- Top 5 frustration sources written into backlog.

Exit gate: do not advance until

Definition of done

The team genuinely wants to replay the graybox race; otherwise retune mechanics instead of moving on.

EXACT GitHub repositories for this part (click to open)

- [R01] arkits/road-rash - Use as historical reference when players say a feature no longer feels like Road Rash.
  https://github.com/arkits/road-rash | RESEARCH ONLY - no source/asset reuse assumed
- [R02] bike-game - Compare basic feedback and pace in another Bevy bike experience.
  https://github.com/Bevy-Jam-5/bike-game | LICENSED EXAMPLE - MIT/Apache code; audit assets separately

BUILD SEQUENCE

Part 10 | Multiplayer feasibility spike

Goal

Prove early that the chosen modern vehicle physics can be reconciled over a network without ruining combat.

Implementation order

- 01. Stand up headless server and two desktop clients on LAN with Lightyear, one graybox course, no online UI or career.
- 02. Implement input packets (tick, vehicle steering/throttle/brake, attack buttons, on-foot movement and remount actions), server-validated simulation and state snapshots.
- 03. Predict the local rider, interpolate remotes and reconcile large errors; send explicit attack events and authoritative results.
- 04. Add 0/50/100/150ms latency emulation, jitter and 0/1/5% packet loss; record visual correction distance and perceived response.
- 05. Run 2 -> 4 players; verify high-speed opposite-direction collisions and punch hit adjudication.
- 06. Decide whether to simplify authoritative physics interactions for riders or use an approach such as server-only ragdolls plus client cosmetic ragdolls.

Engineering specification

- Lightyear 0.31 with Bevy 0.20; input buffers with tick IDs and sequence numbers.
- Initial server authoritative sim 60 Hz; network snapshots target 20-30 Hz; measured bandwidth budget initial target <100 kbps/client in 4-player race.
- Client prediction only for local controller; authoritative collision impulse may require replay correction and conservative thresholds.
- No claim of perfect deterministic rollback across Avian network clients; treat reconciliation as an empirical engineering problem.

Validation / tests

- Two clients connect, race and see each other at 100ms simulated RTT.
- Hit outcome identical across clients and server.
- Packet-loss test produces recoverable play, not disconnect/desync storm.

Exit gate: do not advance until

Definition of done

Networking approach documented with hard performance and correctness measurements; kill or simplify impossible physics assumptions.

EXACT GitHub repositories for this part (click to open)

- [R05] lightyear - Use official client/server, prediction/interpolation and Avian integration examples.
  https://github.com/cBournhonesque/lightyear | DIRECT DEPENDENCY LATER - MIT/Apache
- [R04] avian - Examine physics schedule constraints and contact state serialization.
  https://github.com/avianphysics/avian | DIRECT DEPENDENCY - MIT/Apache

Watch-outs

Networking two physics-heavy bikes is far harder than networking chess pieces. Keep replicated state compact, prioritize safe corrections, and make every gameplay-affecting collision server-owned.

BUILD SEQUENCE

Part 11 | Original art, world-building and asset pipeline

Goal

Replace primitives with an original but unmistakably energetic motorcycle-combat identity.

Implementation order

- 01. Define a ROAD-WORLD art bible first: distinctly readable downtown/town streets, rural farmland, winding mountain/cliff roads, coastal views; then bikes, rider silhouettes, visual density and lighting mood.
- 02. Set Blender -> glTF 2.0 pipeline: coordinate scale, rig names, material conventions, LOD variants, collision prefixes, exports.
- 03. Build modular original road and street kits (lanes, intersections, sidewalks, shoulders, barriers); blockout barns, fields, fences, cows, cliffs, rocks, storefronts, homes, traffic, signage and utility poles BEFORE polishing the first production bike/rider.
- 04. Implement asset importer validation: triangle budgets, skeleton paths, texture sizes, missing material links, content ownership metadata.
- 05. Make a scenic readable art pass that reveals a new landmark every 10-25 seconds; compare frame times and hazard visibility before adding detail or heavy shaders. Every major scenery family gets LOD, collision and audio metadata.
- 06. Create source-asset folders separate from exported runtime assets and optional Blender automation scripts.

Engineering specification

- Asset format: glTF/GLB for 3D, compressed GPU textures, vetted OGG/WAV audio, UI vector/texture assets. Farm animals have lightweight skeletal animation and AI-aware collision; cliffs need safe fall triggers and mesh/physics separation.
- Initial platform targets: bike ~25-50k triangles LOD0; rider ~25-45k; traffic car ~15-35k; dynamic shadow casting tuned per budget.
- Materials PBR; tileable road surfaces; physically plausible light units; avoid relying on costly path tracing as required baseline.
- Provenance file per asset: author, creation date, source file, license, third-party included components, release clearance.

Validation / tests

- Automated validation finds zero missing material/texture refs.
- A town/farm/cliff route has original readable landmarks, cows and roadside hazards, tested at speed; full bike and rider animate through riding/fighting/crash/on-foot/remount transitions.
- Asset licenses complete and credited.

Exit gate: do not advance until

Definition of done

Graybox can be swapped for art without changing physics or race rules.

EXACT GitHub repositories for this part (click to open)

- [R08] bevy - Use scene/asset loader and PBR material APIs.
  https://github.com/bevyengine/bevy | CORE ENGINE DEPENDENCY - MIT/Apache
- [R02] bike-game - Examine original Blender/scene workflow and provenance complexity; do not copy third-party textures without license checking.
  https://github.com/Bevy-Jam-5/bike-game | LICENSED EXAMPLE - MIT/Apache code; audit assets separately

BUILD SEQUENCE

Part 12 | Garage, progression, economy and saves

Goal
Give players a reason to race again while preserving Road Rash-style earned-bike progression.

Implementation order

- 01. Create garage selection, ownership, price, upgrade tiers, stat comparisons and shop UI.
- 02. Implement payout and repair costs; keep repair penalty fair and transparent; tune based on race duration and risk.
- 03. Add rider/loadout cosmetics as separate content items and unlock conditions.
- 04. Serialize career profile, finances, owned bikes, records, unlocks, options and save schema version.
- 05. Implement migration/versioning with safe backup on schema changes; ensure save writes atomic to avoid corruption.
- 06. Build a progression simulator to estimate hours to early/mid/late bikes and detect stuck economy.

Engineering specification

- Contents stored in RON/TOML or JSON for authoring; compiled or validated on load into strongly typed Rust structs.
- Saves in platform-appropriate user data directory, checksummed, rotate backups; never embed raw absolute asset paths.
- Offline economy and online economy have intentionally separate trust models; server verifies online results.

Validation / tests

- No double-credit on restart or crash/quit.
- Save/load equality across 100 write cycles and schema migration fixture.
- Full small career path from starter to upgraded bike playable.

Exit gate: do not advance until

Definition of done
Buying a better bike clearly changes play and repeated races feel rewarding.

EXACT GitHub repositories for this part (click to open)

- [R01] arkits/road-rash - Research classic bike progression, shop pacing and reward loops; rebuild original content only when licensed.
  https://github.com/arkits/road-rash | RESEARCH ONLY - no source/asset reuse assumed

BUILD SEQUENCE

Part 13 | Authoritative multiplayer, lobbies and multiplayer combat

Goal

Convert the verified network spike into robust 4-player races, then plan scaling to 8.

Implementation order

- 01. Add connection lifecycle, join/lobby, ready, bike selection, race start synchronization, disconnects, reconnect policy and results.
- 02. Implement state replication priorities, race-interest culling, interpolation buffers, snapshot deltas and bandwidth/CPU telemetry.
- 03. Add lag-compensated melee against history of server-accepted transforms; reject impossible reach/attack rate.
- 04. Handle client prediction visual smoothing, remote attack animation, cosmetic ragdoll approximation, crash cameras.
- 05. Build server heartbeat, logging, replay/incident dumps and stable race instance ID; replicate rider-foot state separately from abandoned bike state, and validate rider-initiated finish.
- 06. Scale from 4 to 8 users only after measured p95 server tick, client RTT and physics load are acceptable.

Engineering specification

- Authoritative decisions: bike and dismounted-rider transforms, health, weapon pickups, race checkpoints/foot-or-bike finish/payout. Clients never submit authoritative transforms or damage values.
- Server tick budget at 60 Hz = 16.67ms, target <8ms p95 for active session to allow spikes. Start with 4-player race, 8-player stretch.
- Transport use Lightyear-supported UDP/server transport; authenticated identity and basic abuse/rate-limits for public servers.
- Reconnect policy: quick reconnection within 30-60 sec to spectator or rejoin at checkpoint; never retroactively grant placement.

Validation / tests

- Four-player race finishes under 150ms RTT / 1% packet loss with consistent results.
- No duplicate finish/prize event; no client-reported attack can bypass rate limits.
- Measure CPU and egress bandwidth under 8-player stress.

Exit gate: do not advance until

Definition of done

Private 4-player races are consistently playable; 8-player target only after instrumentation confirms headroom.

EXACT GitHub repositories for this part (click to open)

- [R05] lightyear - Protocol definitions and all networking machinery.
  https://github.com/cBournhonesque/lightyear | DIRECT DEPENDENCY LATER - MIT/Apache
- [R08] bevy - Headless Bevy app examples and scheduling.
  https://github.com/bevyengine/bevy | CORE ENGINE DEPENDENCY - MIT/Apache

BUILD SEQUENCE

Part 14 | Content production and modern game expansion

Goal

Expand only now - after baseline riding, combat, career and multiplayer systems have survived playtests.

Implementation order

- 01. Create at least three additional distinctly themed original tracks: dense city streets, open farm country with livestock crossings, dramatic coast/cliff roads, and optional mountain woodland; each needs unique landmarks, hazards and overtaking opportunities.
- 02. Build bike catalog by mechanical classes: lightweight, balanced, touring, superbike, specialty; validate trade-offs.
- 03. Develop rider archetype roster, cosmetics, unlockable loadouts, weapon pickup and special police behavior.
- 04. Add race modes: classic sprint, tournament/career, time trial, private multiplayer, optional combat-heavy arena.
- 05. Implement unlock economy tests and accessibility-friendly difficulty slider.
- 06. Invest in track editor and spawn editor once multiple contributors start adding content.

Engineering specification

- All content uses versioned data schemas and does not require Rust recompilation for tunables; packaged build validates content hash.
- One stable bike spec format covers player, AI and online replication; cosmetics never affect physics without explicit modifiers.
- AI shares the same bikes and physics logic unless surfaced as an intentional difficulty assist.

Validation / tests

- A newly authored bike/track can be loaded without touching core code.
- Cross-class balance tested with 1,000 automated simulated races plus human playtests.
- No softlock in career unlock progression.

Exit gate: do not advance until

Definition of done

Several distinctive tracks and bikes feel unique; no content item breaks the original quality gate.

EXACT GitHub repositories for this part (click to open)

- [R01] arkits/road-rash - Reference original thematic variety and progression structure.
  https://github.com/arkits/road-rash | RESEARCH ONLY - no source/asset reuse assumed
- [R08] bevy - Use scene assets and typed data loader patterns.
  https://github.com/bevyengine/bevy | CORE ENGINE DEPENDENCY - MIT/Apache

BUILD SEQUENCE

Part 15 | Graphics, particles, sound, UI and accessibility polish

Goal

Give the game a contemporary presentation while preserving at-speed legibility and consistent performance.

Implementation order

- 01. Implement scalable lighting and shadows, SSAO/SSR only where sensible, motion blur optional, anti-aliasing controls, reflections and decals.
- 02. Build an auditable FREE-AUDIO asset library, then implement layered RPM/load engine audio, tire surfaces, impacts, melee, police sirens, on-foot footsteps and map-specific ambience (city, farmland, livestock, coast/cliff wind).
- 03. Add optional Hanabi dust, tire smoke, sparks and road debris; keep effects cosmetic and budgeted.
- 04. Polish menus, garage, race HUD, post-race flow, subtitles/captions, colorblind affordances and input rebinding.
- 05. Add quality presets and wide/ultrawide resolution testing; verify clear UI at 1080p/1440p/4K.
- 06. Build local replay camera or highlight recorder only if core performance remains healthy.

Engineering specification

- Performance requirement: 1080p high 120fps aspirational on RTX 3080; mandatory baseline 1080p medium 60fps stable at 4 players / busy traffic; targets not promises.
- Particles capped by quality mode; texture budgets monitored; GPU frame-time P95 and streaming spikes included in profiling.
- Never use film grain/motion blur or violent camera shake as mandatory visuals.

Validation / tests

- Core HUD readable at high speed and with varied FOV.
- No text-only critical information is conveyed exclusively by color.
- Audio dynamic range/input settings survive reload; engine RPM smoothly crossfades; farm/city/cliff ambience transitions smoothly and sources have valid release permissions.

Exit gate: do not advance until

Definition of done

Visual quality and audio clarity are strong enough for external trailers without losing performance.

EXACT GitHub repositories for this part (click to open)

- [R07] bevy_hanabi - Optional GPU particle systems and profiling patterns.
  https://github.com/djeedai/bevy_hanabi | OPTIONAL DIRECT DEPENDENCY - MIT/Apache
- [R08] bevy - Rendering, lighting, post-processing and UI APIs.
  https://github.com/bevyengine/bevy | CORE ENGINE DEPENDENCY - MIT/Apache
- [R09] bevy_kira_audio - Optional audio busses, spatial panning, crossfade and per-effect controls; 0.27 with Bevy 0.20.
  https://github.com/niklasei/bevy_kira_audio | OPTIONAL AUDIO DEPENDENCY - MIT/Apache

BUILD SEQUENCE

Part 16 | Performance, QA, packaging and release

Goal

Build a stable release candidate, manage rights, and ship Windows/Linux distributions with measurable reliability.

Implementation order

- 01. Profile server tick, GPU time, draw calls, render stages, animation cost, physics contacts and network bytes per client.
- 02. Implement automated functional tests, golden replay-like state checks, soak tests, reproducible crash logs and bug template.
- 03. Run OS test matrix (Fedora KDE Wayland/NVIDIA, Windows/NVIDIA/AMD where possible), controller matrix and ultrawide HUD tests.
- 04. Do localization-ready text and accessibility review; audit sound/image/model licenses; finalize privacy, telemetry and moderation policy.
- 05. Package release builds, smoke-test clean installs, configure Steam depots/achievements/cloud saves only if desired and feasible.
- 06. Plan staged closed test -> public demo -> launch; build hotfix and support process; capture crash rates and retention.

Engineering specification

- CI: format, lint, tests, artifact license audit, content validation and cargo audit/deny where compatible.
- Automated 30-minute race soak with 8 bots (if capacity target remains 8); multiplayer soak with high-latency/loss injection.
- Crash-free session target >99% in test population; P95 renderer frame-time <=16.7ms for 60fps preset; p95 server tick <=8ms.
- Public title and branding must avoid EA trademark conflict unless licensed; clear written commercial-use rights for all original work.

Validation / tests

- No P0/P1 bugs open, release checklist signed, all references in SBOM/license inventory.
- Clean PC installs, network port tests, controller reconnect, save migration pass.
- Post-release hotfix process rehearsed.

Exit gate: do not advance until

Definition of done

Ship candidate satisfies functional, performance, stability and legal criteria; release is a conscious decision, not simply a build succeeding.

EXACT GitHub repositories for this part (click to open)

- [R08] bevy - Official platform packaging examples.
  https://github.com/bevyengine/bevy | CORE ENGINE DEPENDENCY - MIT/Apache
- [R05] lightyear - Network logging and transport considerations.
  https://github.com/cBournhonesque/lightyear | DIRECT DEPENDENCY LATER - MIT/Apache
- [R04] avian - Physics diagnostics and limits.
  https://github.com/avianphysics/avian | DIRECT DEPENDENCY - MIT/Apache

ROADSIDE ENVIRONMENT PLAYBOOK

Design priority | the world is the game

World-first requirement

The scenery is NOT set dressing. The feeling of racing through living towns, working farms, windy cliffs and irregular streets is a primary gameplay pillar, equal in priority to bike handling and melee. A road with random trees is insufficient.

Five environment promises

World region	Readable roadside features	Playable consequences
Town / city	Shopfronts, houses, intersections, lights, parked cars, sidewalks, alleys, road signs	Traffic intersections, hard corners, visibility blockers, sidewalk curb hits, police density
Farmland	Open fields, red barns, tractors, silos, grazing cattle, wooden fencing, dirt entrances	Crossing livestock, tractor traffic, low-grip dirt shoulders, fence impacts
Cliff / mountain	Hairpins, ledges, retaining walls, valleys, rockfall, viewpoints, guardrails	Unprotected drops, tight sightlines, dangerous downhill braking, rock hazards
Coastal highway	Ocean vistas, coastal towns, bridges, dunes, pull-offs, gulls	Wind ambience, changing road grip, road-edge risk, dramatic scenic reveals
Woodland	Dense trees, cabins, shadows, fallen branches, deer, gravel turnoffs	Occlusion, night/day contrast, animal crossings, narrow shoulders

Road corridor pipeline - not a huge open world

- Stage E0: Author road spline and lane widths, shoulders, sidewalks and vertical curves; place checkpoint and finish volumes.
- Stage E1: Define zones at road stations (distance along route), e.g. Town 0-2 km, Farmland 2-5 km, Cliff 5-8 km; use smooth art/audio transitions.
- Stage E2: Stamp parametric roadside lots and terrain cells along route; reserve 10-30 m of critical view space around bends and intersections.
- Stage E3: Populate distinct landmarks (barn, gas station, bridge, cow pasture, cliff vista) and interactive obstacles (fence, cow, car, rock).
- Stage E4: Build collision proxy and navigation surfaces for both bikes AND on-foot riders. No opaque "scenery only" barriers in the walkable corridor.
- Stage E5: Stream/LOD per distance and visibility. Preserve consistent hazards even if high-detail visual meshes unload.

World encounter authoring - data contract

```
WorldZone(
  id: "farm_crossing_03", from_m: 2550.0, to_m: 3410.0,
  biome: Farmland, road_style: TwoLane, speed_profile: FastRolling,
  scenery_sets: ["barns", "silos", "fences", "fields", "cows"],
  ambient_audio: "farm_day", music_intensity: 0.5,
  encounters: ["tractor_merge", "cow_crossing_optional"],
  road_hazards: ["ditch", "fence_post", "mud_shoulder"],
  safe_spawn_regions: ["shoulder_left", "shoulder_right"]
)
```

Animal encounters and cliff rules

- Cows: grazing/idle, startled, turn, walk/cross, collision and reaction states. Make crossing readable with fencing, gates and sightline cues; do not random-spawn an unavoidable cow in front of the player.

- Physics: cows need only simple capsules or convex proxy colliders; ragdolls are optional cosmetic behavior. Respect welfare-themed tone and age ratings if collisions are graphically depicted.
- Cliffs: camera and road signs telegraph a drop; support intentional high-risk shortcuts only when course rules explicitly permit them. Fallen riders receive rescue/recovery with penalties rather than soft-locking below terrain.
- City: intersections require traffic priority, curb collision types, separate sidewalk physics, roadside walls with proper ground-level collision.
- Farm: tractor slow movers, gravel driveways, dust trails, hay bales, utility poles, fencing, grazing animals and ambience form a visually different rhythm.

Scenery-performance budget, provisional

Asset family	LOD / simulation plan	Collision rule
Buildings	Modular fronts + distant cards/low LOD; interior not needed for v1	Simple boxes; no per-window collision
Trees/rocks	LOD and instancing; GPU culling where applicable	Proxy only near rideable/foot corridor
Cows / animals	Stateful agents near camera; far grazing can animate cheaply	Server-owned critical collisions
Cliff rocks/barriers	Tiled terrain, mesh decimation and surface shaders	Separate stable low-poly collision
Road furniture	Batched signs, fences, guardrails; proximity pop controls	Relevant posts/rails have collision

Environment acceptance

On a ~3-minute race at 100-220 km/h, a tester should identify where they are without looking at the HUD. Target a memorable silhouette or encounter every 10-25 seconds. No scenery pop-in hides an active collision hazard. Walking after a crash must still work near fences, barns, curbs and cliffs.

KEEP THE CLASSIC ODDITY, DELIBERATELY

Technical spec C | dismounted rider control

Historical note: an original Genesis Road Rash guide explicitly documents crossing the finish line on foot, while an older game manual describes automatically running to the bike with player steering available [H01]-[H02]. Confirm exact behaviors and control bindings for the 1996 Windows edition separately. The following is the requested new-game behavior, not a claim that every original platform behaved identically.

Player promise

After a crash you are a controllable character, not a homing missile: you may run toward the bike, run away from it, explore the shoulder, dodge cars, cross the finish line on foot, or choose to remount. Left alone, your rider automatically heads for the bike as a helpful default.

State machine - separate rider and bike entities

```
Riding
  | impact / knocked off
  v
Ragdoll -> GettingUp -> OnFootAutoSeekBike <-> OnFootManual
                            | proximity + safe remount
                            v
                        Remounting -> Riding
                \_ finish valid -> FinishedOnFoot
                \_ police / out-of-world -> BustedOrRecover
```

Input precedence and camera behavior

- Manual overrides assist: directional analog/WASD input above deadzone immediately disables bike-seeking movement. Camera-relative walk input gives independent 360-degree movement, not just forward/back-to-bike.
- Assist returns only after input released: 0.25-0.5 seconds of neutral input (prototype tunable) resumes walk toward the bike using an obstacle-aware waypoint path; player can cancel again at any moment.
- No compulsory snap: auto-seek approaches bike, but auto-remount only within a safe interaction radius, at low bike speed, and after collision/animation checks. Prefer manual remount affordance option in accessibility settings.
- Camera: while on foot follow rider, show bike direction icon and distance; if player runs away, never drag camera to bike. At finish, cut to victory/results even though bike remains behind.
- Motion: ground run speed initial 4.5-6.5 m/s (proposed), predictable start/stop, step-up/terrain slope support; rider should not get stuck behind a waist-high farm fence.
- Risk: civilian traffic and approaching rivals still collide with or can endanger rider; implement readable collision feedback and avoid punishing repeated spawn kill loops.

Server-authoritative finish contract

```
Racer { racer_id, rider_entity, bike_entity, last_checkpoint,
        last_route_progress, status, finish_tick }

On crossing a finish plane, accept candidate entity if:
  1) it is this racer's RIDER or BIKE entity,
  2) racer status is Racing,
  3) all required ordered checkpoints are valid,
  4) crossing is in forward direction with swept detection,
  5) racer finish event has not already been recorded.

emit RaceFinished(racer_id, source=OnFoot|Bike, tick)
result is based on racer, NOT bike entity position.
```

Test matrix - all mandatory

Test	Expected outcome
Crash 5 m before finish; run forward on foot	Racer finishes, results generated once
Crash 5 m before finish; run away from bike across line	Manual control wins; foot finish valid
Crash; neutral controls	Rider automatically seeks bike, avoids known obstacle
Crash; hold left/right/back, then release	Moves chosen direction; assist resumes when neutral
Bike crosses while rider crosses in same tick	Exactly one finish payout and event
Walk backward through finish / skip checkpoint	No result until route progress valid
Run off a cliff / become trapped between fence and car	Recovery possible; no invisible wall softlock
Disconnect while on foot; reconnect or AI timeout	Authoritative consistent rider/bike states

Production milestones

- Part 04: allocate on-foot input context (WASD/stick) and chase camera rules.
- Part 06: prototype ragdoll -> get-up -> manual running -> optional auto-seek -> remount.
- Part 07: make finish-line trigger accept either rider or bike after checkpoints.
- Part 08: AI dismounted recovery and traffic avoidance.
- Part 10/13: replicate rider state and bike state separately, validate foot finish on server.
- Part 11: test walk collision with all environment zones, especially farm fences and cliff edges.

COMMERCIAL-USE SOUND PLAN

Technical spec D | free audio pipeline

Audio is gameplay

Engine load, grit beneath tires, rider impact, cow lowing in the fields, sirens echoing between buildings, and wind against the coastal cliff must tell the player what the road is doing. Free source files still need rights verification and editing.

Free sources, exact responsibilities and caveats

Source	Use in game	Rights / implementation caveat
Mixkit motorcycle [A01]	Idle, revs, shift transients, accelerations, pass-bys	Custom Free SFX license permits use in video games; cannot redistribute as stock
Kenney Impact [A02]	Punches, kick impacts, crash thuds, roadside strikes	CC0, 130 files; layer/eq for weight, avoid monotonous repeats
Kenney UI [A03]	Menu controls, timer, checkpoint sound	CC0; no attribution required
Sonniss GDC 2026 [A04]	Foley, impacts, environmental ambiences and vehicle detail	Custom commercial license; no standalone resale or AI training
OpenGameArt CC0 track [A05]	Rock racing music prototype	CC0 per specific track; screen each song individually
Poly Haven [A06]	Not sound: CC0 scenery/terrain textures/HDRIs	Use in Blender/Bevy for roads, barns and cliffs

First-pass audio event inventory (proposed ~36 clips)

Group	Essential prototype samples	Hook
Motorbike (10)	Idle loop, 2 mid loops, high loop, rev hit, shift up/down, 2 pass-bys, starter	Bike engine RPM, throttle, gear, distance
Surface (6)	Asphalt hiss, gravel, dirt, grass, skid, wind	Wheel slip and `SurfaceType`
Combat (5)	Punch light/heavy, kick, weapon impact, rider grunt	AttackResolved / HitReaction
Crash (5)	Body fall, metal impact, scrape, fence break, glass/crumple	CrashStarted / ContactSeverity
World (6)	City bed, farm insects/wind, cattle moo, tractor, cliff wind, coast/surf	WorldZone and entity position
Police/UI (4)	Siren near/far, checkpoint, menu / finish jingle	PoliceState / RaceState

Runtime audio architecture

- Bike source: 3-4 seamlessly looping RPM bands, envelope crossfades and subtle pitch adjustments from actual simulated rpm; add throttle/transmission and gear-change one-shots. Avoid recording-only playback tied directly to throttle button.
- Listener: listener at chase camera, physical emitters at world bikes, police, cows and environmental landmarks. Use distance rolloff and stereo field carefully; expose a minimum-volume setting for opponent readability.
- Environment: blend city traffic hum, farmland insects/wind and cow calls, coastal wind/waves across zone boundaries; local 3D sources punctuate large non-positional ambience beds.
- Combat: randomize among 3-5 permitted variants; layer body and metal sounds in response to actual validated events; no duplicate emission during client prediction reconciliation.
- Music: queue properly licensed tracks separately from SFX, keep driving music ducked during critical gameplay cues, provide streamer-friendly mute option.

- Implementation: start with Bevy audio for loop/volume/pitch and verify exact capabilities in pinned version; evaluate bevy_kira_audio 0.27 [R09] if dynamic crossfade, mixer and bus control demand it. Keep an `AudioService` boundary.
- Asset ledger: path, author, source URL, exact license at acquisition, license copy, hash, modifications, permitted distribution use, attribution field, approval timestamp.

QA acceptance

- Bike pitch, gain and loop blend remain smooth across idle-to-redline; no audible click at repeat boundaries.
- Three location ambiences are distinguishable without looking at a map; livestock vocalization matches visible source.
- Two motorcycles at different distance/gear remain perceptually distinct; a punch and crash never fire twice after network correction.
- Master, engines, SFX, ambience and music sliders persist; captions cover vital non-speech signals; streamer-safe playlist exists.
- No proprietary original Road Rash music, voices or sound effects in distributed build.

Audio integration order and sample content layout

```
assets/audio/
  engines/{class_id}/{idle,mid,high,gearshift}.ogg
  foley/{footsteps,crashes,combat,surfaces}/
  world/{town,farm,coast,cliff}/
  music/{licensed_track_id}.ogg
content/audio_manifest.ron  # source link, author, license, hash
```

Milestone	Trigger / data contract	Required proof
P03 bike controls	RPM / throttle / gear -> engine loop weights	Accelerate and brake without loop gap
P05 combat	Confirmed AttackResolved -> material + impact variation	No doubled predicted/network sound
P06 on foot	Ragdoll, footsteps by surface, remount interaction	Ground sound matches foot position
P11 scenery	WorldZone mix + nearby cow/tractor/traffic emitters	Town, farm and cliff sound different
P13 network	World SFX only for confirmed events; local latency cues separated	Lag does not replay crashes
P15 mix	Engine/world/combat/music busses and accessibility captions	Reference loudness and clipping checks

Recommended tool decision

For the first vertical slice, the built-in Bevy audio pipeline is enough to prototype. If you need real mixing busses, effects and precise crossfades, the MIT/Apache bevy_kira_audio 0.27.x adapter supports Bevy 0.20 [R09]. Avoid switching audio backends mid-production without test coverage.

IMPLEMENTATION CONTRACT

Technical spec A | motorcycle dynamics

Physics strategy and mathematical model

Recommended first iteration uses one dynamic bike chassis, two suspension contacts, game-controlled steering/lean assist, slip-limited tire forces and visual wheel/rider animation. Avian resolves rigid-body collisions. A more advanced two-wheel rigid-body model is optional after the arcade controller meets the fun gate.

Parameter	Starter range / representation	Notes
Simulation frequency	60 Hz fixed; optional substeps	Do not bind motion to FPS
Bike mass	150-290 kg dry + rider (tunable)	Proposed class ranges, not canon
Engine	Torque curve + redline + gear ratio	Can begin with throttle-to-force
Grip	Surface mu: asphalt 1.0, gravel 0.65, grass 0.4	Proposed tuning defaults
Lean assist	Desired roll vs speed / steering	Clamp to controllable lean
Suspension	2 wheel contact rays + spring-damper	Use force at contact location
Bike damage	Integrity + local impact severity	Keep accessible not overly realistic
Crash	Impulse threshold + rider balance	Gamey thresholds allowed
Collision	Chassis capsule/convex + CCD on fast bodies	Collision simplification for stability

Suggested implementation order and acceptance metrics

- M0: kinematic movement on spline; reach finish.
- M1: dynamic root body and upright assist; survive hairpin.
- M2: front/rear wheel raycast contacts, braking and speed curve.
- M3: surface friction, slip, leaning, controlled drift.
- M4: collisions and knockoff trigger integration.
- M5: user playtest with three distinct bike presets and controller.

Technical design guidance

Do not simulate lateral and longitudinal tire slip as two arbitrary independent clamped accelerations; resolve the combined force budget so high braking reduces available turning force. Use a simple traction circle first, then a richer curve only if the extra realism improves predictability. Side impacts should cause meaningful loss of balance but not a coin-flip crash every contact.

Diagnostics to record each tick

```
bike_id, tick, dt, pos_xyz, vel_xyz, speed_mps,
steer_input, throttle, brake, yaw_rate, lean_deg,
front_contact, rear_contact, spring_force_f, spring_force_r,
slip_lat, slip_long, mu_surface, damage, crash_state
```

Engineering risk to track

Full-physics leaning + high-speed collisions + multiplayer prediction is the hardest integration area. Keep custom bike forces isolated behind `VehiclePhysicsAdapter`; design event-driven gameplay to work even when client physics differs slightly from server physics.

AUTHORING SCHEMA

Technical spec B | combat, crash & AI

Typed core data examples

```
# content/weapons/chain.ron  (illustrative schema, not final Rust)
WeaponDef(
  id: "chain", hand: "right", range_m: 1.35,
  startup_ms: 210, active_ms: 140, recovery_ms: 420,
  stamina_cost: 14, damage: 25, stagger: 0.40,
  knockback_impulse: 175.0, pickup_allowed: true
)

# content/bikes/balanced_01.ron
BikeDef(
  id: "balanced_01", class: "balanced", mass_kg: 216.0,
  top_speed_kmh: 235.0, torque_scale: 1.0,
  grip_asphalt: 1.0, lean_assist: 0.78,
  durability: 100.0, visual_asset: "bikes/balanced_01.glb"
)
```

Values above are proposed starting tunables. They are not extracted Road Rash stats and require playtesting. Store tunables in separate authoring files so adjustments do not change source code.

Event API

```
struct AttackIntent { rider_id, attack_id, input_tick }
struct AttackResolved { attacker, victim, tick, damage, impulse }
struct CrashStarted { rider_id, bike_id, cause, position, impulse }
struct RecoveryCompleted { rider_id, checkpoint_id, tick }
struct RaceFinished { racer_id, race_id, finish_tick, rank }
struct PoliceBusted { racer_id, trigger, tick }
```

AI architecture

Layer decision-making (a small finite-state machine or utility selector) above the same player vehicle controller. Planning chooses desired speed, line, target rider and attack timing; the input layer translates that into steering/throttle/attack controls. This keeps AI from circumventing the vehicle simulation. Limit expensive perception updates to 5-15Hz while control stays 60Hz.

Agent	Intent priorities	Failure/recovery
Racer	Follow line, overtake, evade, fight, finish	Avoid impossible overtakes; recover from crashes
Brawler	Shadow opponent, get side-by-side, attack	Back off when stamina low
Traffic	Follow lane, maintain gap, turn	Brake or path reroute on obstruction
Police	Patrol, assess violation, pursue, intercept, bust	Lose target, return to patrol

PACKETS, TICKS AND AUTHORITY

Technical spec C | networking

Message categories

Direction	Payload	Reliability
Client -> server	InputBatch: tick IDs + steer/throttle/brake/attack	Unreliable-sequenced; redundant recent inputs
Server -> client	Snapshot: authoritative bike pose, velocity, health, race progress	Unreliable-sequenced delta snapshots
Server -> client	Events: hit, crash, police bust, weapon pickup, finish	Reliable ordered (bounded) or deduplicated event IDs
Client <-> server	Lobby/ready/selection, session join/leave	Reliable ordered
Server -> client	Race result/payout and save entitlement	Reliable and idempotent

Ownership

- Client can request throttle/steering/attack; server decides positions, collisions, damage and progression.
- Server validates rate and magnitude of inputs, bike selection, weapon ownership, attack cooldown and finish checkpoint sequence.
- Each state has tick number; clocks use offset estimation for interpolation.
- Remote riders interpolate between buffered snapshots; local rider predicts steering/acceleration and eases toward corrections.
- Ragdolls and particle FX can be mostly cosmetic, but crash start/end and recovery are authoritative.

Loss and latency test schedule

Scenario	Pass condition	Action if failing
LAN 0-5ms	No visible jitter; accurate hits	Fix physics ownership / transforms
50ms RTT / 1% loss	Smooth and fair	Tune interpolation buffer
100ms RTT / 2% loss	Playable fight proximity	Tune reconciliation / lag compensation
150ms RTT / 5% loss	Degraded gracefully; no catastrophic desync	Bound correction and improve messaging
8-player stress	p95 server tick below target	Simplify replication / physics / crowd

Known hard problem

A conventional Avian physics world is not guaranteed bit-identical across machines, especially with differing order/precision, contact ordering and ragdoll constraints. Avoid designing a protocol that requires perfect lockstep across arbitrary PCs. Reconciliation/snapshot-based authority is the safer initial assumption.

ASSET AUTHORING RULES

Technical spec D | content, rendering, tools

Original track content bundle

```
content/tracks/coastal_01/
  track.ron              # spline control points, widths, friction
  checkpoints.ron        # start / ordered checkpoints / finish
  spawn_points.ron       # AI / traffic / police spawn groups
  props.ron              # authored placement IDs + LOD
  art/                  # original Blender source (not shipped)
assets/tracks/coastal_01.glb
assets/tracks/coastal_01_collider.glb
```

Asset limits and visual-performance budget

Area	Proposed guardrail	Implementation measure
Track	1 scene chunk per 100-300m initially	Cull out-of-view chunks; stream ahead
Bike	25-50k triangles LOD0	LOD down / instancing where possible
Rider	25-45k triangles LOD0	Animation budget and mesh LOD
Car	15-35k triangles LOD0	Limit physics complexity for traffic
Textures	2K bike/character primary; 1K secondary	GPU memory and streaming telemetry
Shadows	Limited cascades, traffic distance cutoff	Shadow cost profile per preset
Effects	Max particle emitters by quality	GPU pass cost and overdraw

Guardrails are adjustable production targets. The visual target should be established by measured GPU budgets, not by a fixed triangle count alone. On the RTX 3080 development machine, create repeatable camera sequences and record 1080p/1440p frame-time percentiles.

Research catalog schema: never confuse canon with implementation

Record section	Fields
Evidence	source_url, build_version, observation_note, confidence, date
Classic reference	display_name, observed_color, approximate stats, original source
New design	our_id, class, visuals, new stats, unlock rules, creator
Rights	owner, license, derived_from, approval_status, attribution
QA	validator_version, last_tested, failing_cases

NON-FUNCTIONAL SPECIFICATION

Release requirements and targets

Category	Baseline / target	How measured
Rendering	1080p medium >=60 FPS; aspirational 1080p high 120 FPS on RTX 3080	P95 frame-time; representative race with traffic
Simulation	60Hz fixed; p95 gameplay + physics <8 ms on server target	Engine timers, telemetry, consistent reproducible scenario
Netcode	4 players launch; 8 stretch; 100ms RTT / 2% loss playable	Injected packet jitter/loss; combat results checked
Input	Gamepad + keyboard; remappable, deadzones	Manual QA matrix; automated action tests
Content	All runtime assets have provenance and legal clearance	Automated export audit + manual rights review
Save reliability	No corrupted saves under interrupted writes	Atomic save tests / backup restoration
Crash stability	>99% crash-free sessions target during beta	Telemetry opt-in and crash reporting
Accessibility	FOV/blur/shake options, subtitles, rebindings	Accessibility checklist with real testers
Platform	Native Linux and Windows playable builds	Clean machine install + controller + GPU matrix

Planning estimates - solo developer, not commitments

Assuming a capable Rust programmer working part-time, using existing Blender assets only when appropriately licensed: initial offline physics prototype 4-8 weeks; enjoyable vertical slice 3-6 months; online prototype + first content art 3-6 additional months; a credible polished multi-track public release 12-24+ months. A single developer making every model, animation and soundtrack from scratch could take substantially longer. Treat this as risk planning, not a promised schedule.

Scope cut options

- Ship first with 4 riders, not 16; increase only if stress results demand it.
- Start with one environment style, not a continent-scale open world.
- Use readable stylized PBR before expensive realism.
- Only add 3D ragdoll joints needed for fun; cosmetic simplification is acceptable.
- Use a fixed workshop/garage, not open-world pedestrian AI.
- Treat online career unlocks, crossplay and modding as post-launch opportunities unless the first playtests require them.

RELEASE SAFETY

License and IP playbook

Two distinct release paths

Path	Allowed by default	Still requires review
Original spiritual successor	Original code, assets, brands, new roads; motorcycle-combat game mechanics	Trade dress and similarity; music, fonts, real brand names, contributor licenses
Faithful Road Rash remake	No permission assumed to use EA-specific protected work	Road Rash mark, original art/music/levels/characters and any license issues

Per-repository treatment

- Bevy / Avian / Lightyear / Enhanced Input / Hanabi: permissive code licensing documented; retain required copyright and license notices and any patent/license conditions.
- Crazy Bike: README grants MIT/Apache licensing to code, but third-party art/music and the Bevy logo have different rights. Copy only clearly authorized files and attribute appropriately.
- DriftMaster: public source does not prove a reusable license. Research only unless a license is verified or creator grants permission.
- arkits/road-rash: research only; its independently written code may have its own copyright and its references to EA data do not convey EA permissions.
- Original game files: do not bundle retail binaries, original sprites, audio, models, save files with protected content or extracted commercial data in a public distribution without legal clearance.
- New free sounds: commercial use is not the same as unrestricted redistribution. Keep source EULAs and source-specific licenses in the rights manifest; no EA original soundtrack/audio.

Not legal advice

This document gives practical risk controls, not a legal opinion. Obtain qualified IP counsel review before shipping under the Road Rash name or reproducing EA-specific expression. Copyrighting your own Rust code does not grant rights to earlier protected assets.

Rights clearance checklist

- Written commercial-use permission for every source asset and contribution.
- Proof of ownership for soundtrack, voice, logos and real motorcycle brand usage.
- Third-party attribution file generated from current lockfile and provenance manifest.
- No unlicensed material in installers, trailers, game website or screenshots.
- Trademark review of the final game title and marketing design.

PRACTICAL SOLO STARTING ORDER

First 30 work sessions

Sessions	Concrete target	Do not advance without
01-02	Create workspace, CI, debug metrics, clean scene	Launches Fedora and tests pass
03-04	Create spline road with mesh/colliders, course checkpoint gates	Driveable route + start/finish detected
05-08	Bike acceleration/brakes, two wheel raycasts, camera	Stable turning and braking
09-10	Lean assist, surface grip and three tuning presets	Each bike feels different
11-12	Track timer/position/results and quick reset	Complete loop works
13-15	Punches/kicks plus hit / stagger state	Combat timing and range verified
16-18	Crash, on-foot free movement, assist-to-bike, foot finish	Manual direction works; no softlock; finish valid
19-22	Three AI opponents and traffic budgets	Repeated fun full race
23-24	Police states, town/farm/cliff audio zones, placeholder SFX	Threat pressure and scenery feel distinct
25-27	Ten playtests, ride tuning, metrics, bug fixing	Game fun gate passed
28-30	Lightyear spike: two clients + server, input/prediction	Reproducible 100ms test and notes

Suggested GitHub issue template

```
Title: [P03] Add speed-sensitive steering with lean assist
Why: Bike must corner at 160 km/h without oscillation
Inputs: VehicleControls + surface metadata
Outputs: stable yaw/lean + telemetry trace
Acceptance:
 - Hairpin achievable without tipping unexpectedly
 - 60Hz fixed-step result consistent at 30/120 render FPS
 - 10-minute aggressive driving soak no NaNs
Evidence: video clip + data trace + test name
Dependencies: P02 collision road; P03 wheel contact
License: no external code/assets copied
```

Exit artifacts after every part

- A tagged build, test results, sample gameplay recording and performance metrics.
- An updated `docs/decisions/` entry for the new architectural assumptions.
- An explicit list of bugs and limitations; do not silently move untested requirements forward.
- A source/rights note for each dependency or asset introduced.

Final action sequence

Build an interesting road-world and bike handling together. Parts 05-09 must prove punches, crashes, manual on-foot movement, auto-return assist and on-foot finishing in a scenic town/farm/cliff race. Record source rights for every sound and asset. Run network feasibility before scaling world art; release only after rights and QA gates pass.

AUDITABLE REFERENCES

Sources and repository links

All links below were checked against public repository pages, release/docs.rs records, or project READMEs as of October 30, 2026. Source status can change; pin a specific commit/tag in the build and recheck licenses before copying code.

R01 | Road Rash native port (research only) https://github.com/arkits/road-rash - Reverse-engineered C++/SDL3 1996 Windows game; gameplay/asset research, not implementation base.

R02 | Crazy Bike / Bevy Jam 5 https://github.com/Bevy-Jam-5/bike-game - Open source bicycle game, README specifies MIT/Apache code; mixed third-party asset licenses.

R03 | DriftMaster vehicle dynamics https://github.com/blyfoten/driftmaster - Bevy 0.14 + WebGL rally physics and telemetry; no explicit repository license verified.

R04 | Avian physics https://github.com/avianphysics/avian - Avian 0.8 / Bevy 0.20, docs.rs dependency confirms compatibility.

R05 | Lightyear networking https://github.com/cBournhonesque/lightyear - Lightyear 0.31 / Bevy 0.20, server authoritative examples.

R06 | Bevy Enhanced Input https://github.com/simgine/bevy_enhanced_input - 0.27 supports Bevy 0.20, licensed MIT/Apache.

R07 | Hanabi GPU particles https://github.com/djeedai/bevy_hanabi - Main branch compatibility with Bevy 0.20 per README.

R08 | Bevy engine https://github.com/bevyengine/bevy - Official engine examples; 0.20 release Oct 8, 2026.

R09 | Bevy Kira audio https://github.com/NiklasEi/bevy_kira_audio - Audio mixer alternative: version 0.27 for Bevy 0.20, MIT/Apache.

S01 | Bevy 0.20 release notes https://bevy.org/news/bevy-0-20/ - Official feature and version information.

S02 | Avian3D 0.8.0 docs https://docs.rs/crate/avian3d/0.8.0 - Avian API and supported Bevy version.

S03 | Lightyear 0.31.0 docs https://docs.rs/crate/lightyear/0.31.0 - Network API and supported Bevy version.

S04 | Bevy Enhanced Input README https://github.com/simgine/bevy_enhanced_input - Version mapping and license.

S05 | Hanabi README https://github.com/djeedai/bevy_hanabi - Compatibility and licensing.

S06 | Bevy 0.20.1 release https://github.com/bevyengine/bevy/releases/tag/v0.20.1 - Patch-version verification.

H01 | Road Rash Genesis GameFAQs walkthrough https://gamefaqs.gamespot.com/genesis/586425-road-rash/faqs/55386 - Confirms on-foot crossing of finish line and motorcycle follow oddity; edition-specific.

H02 | Road Rash original manual text https://www.abandonwaredos.com/docawd.php?idg=2766&sf=roadrashmanual.txt&sg=Road+Rash&st=manual - Describes automatic run to bike with player left/right steering; confirm version-specific controls separately.

A01 | Mixkit free motorcycle SFX https://mixkit.co/free-sound-effects/motorcycle/ - Motorcycle samples; verify each file and Mixkit Free SFX license.

A02 | Kenney Impact Sounds https://kenney.nl/assets/impact-sounds - 130 CC0 impact/foley sounds.

A03 | Kenney Interface Sounds https://kenney.nl/assets/interface-sounds - CC0 interface audio pack.

A04 | Sonniss GDC 2026 free sounds https://gdc.sonniss.com/ - Commercial sound library with custom license and no AI training.

A05 | Super Wreck Roadway music https://opengameart.org/content/super-wreck-roadway - CC0 original rock track, downloadable loop and outro.

A06 | Poly Haven CC0 license https://polyhaven.com/license - CC0 HDRIs, textures and models for roadscapes.

A07 | Mixkit sound effects license https://mixkit.co/license/modal/sfxFree/ - Permits game SFX use, no standalone redistribution.